How It All Fits Together

The shape of the LearnPerk system, and how we will know model switching was worth building

Prepared by Anagha

What is in this document

Two halves. The first draws the system. The second says how we will judge it.

1 The whole system on one page

One picture: the phone, the cloud, and what travels between them.

2 What each part does

The same picture, walked through block by block.

3 What comes from the cloud

Aggregated, anonymous information — and what is deliberately not sent.

4 One decision, end to end

The train enters a tunnel. What happens in the next fifth of a second.

5 How we will compare it

Four versions of the app, run against the same journeys.

6 The metrics

Six families of measurement: energy, quality, never-blank, the switching itself, the person, and fairness.

7 What a result looks like

The four conditions that have to hold before we call switching a win.

8 Five ways to get this measurement wrong

Including the one that would make our numbers look great and be untrue.

9 Words we used

A short glossary.

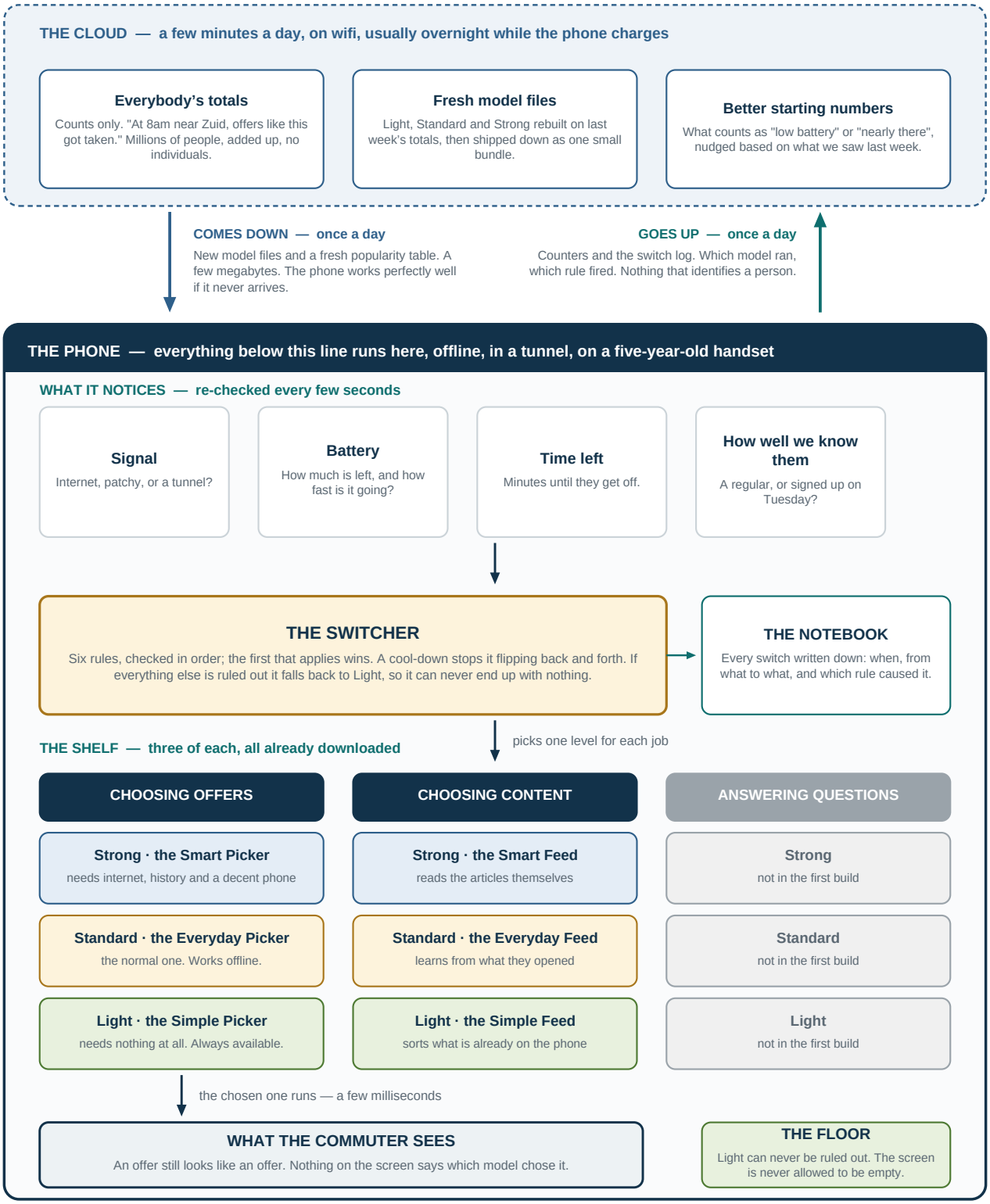
The short version, if you read nothing else

Three models of each kind sit **on the phone**. A small piece of software watches four simple things — signal, battery, time left, how well we know the person — and picks which model to use, changing its mind a few times per journey. The **cloud** is not in the loop: once a day it sends down fresh model files and a popularity table built from everybody’s behaviour with the names stripped out, and that is the whole of its involvement.

To prove it was worth doing we run the same journeys four ways — always Light, always Standard, always Strong, and switching — and compare them on **energy**, **quality**, **how often the screen had something on it**, and **what the switching itself cost**.

1 The whole system on one page

Everything inside the big box happens on the commuter's phone, with no internet needed. The cloud sits above it and is only involved once a day.



The four boxes at the top feed the switcher; the switcher picks one row off each shelf; the picked model produces what the commuter sees. The cloud never sits in the middle of a decision — if it vanished for a week, everything below the navy line would carry on working.

2 What each part does

The same picture, one block at a time. Read the middle column if you only want to know what it is for; read the right one if you want to know what it actually is.

The block	What it is for	What it actually is
What it notices the four boxes at the top	Tells the app what kind of moment this is. Four readings, refreshed every few seconds.	A small background job reading things the phone already reports — connectivity state, battery level and drain rate, the GVB arrival estimate, and a count of how much history we hold for this person. Nothing new is collected.
The switcher the amber box in the middle	Turns those four readings into a decision: which level each job should use right now.	Six ordered if-then rules, a minimum dwell time before it is allowed to change its mind again, and a hard floor that guarantees Light is always a legal answer. A few hundred lines of ordinary code — no model involved in the switching itself.
The shelf the nine coloured chips	Holds every model the app might want, already downloaded, ready to run.	Three model files per job, bundled with the app and refreshed from the cloud. Under 2 MB for the whole offer ladder. They are loaded lazily and the unused ones are dropped from memory.
The notebook the small box on the right	Makes the switching explainable. If we cannot say why a switch happened, we treat that as a bug.	A local append-only log: timestamp, job, level before, level after, the rule that fired, and the four readings at that instant. Uploaded in aggregate, kept on the phone in full.
The floor the green box, bottom right	The promise that the screen is never empty.	Not really a component — it is a constraint the switcher is written to obey and the tests check. The Light models are built to need nothing: no network, no history, no meaningful battery, no modern chip.
The cloud the dashed box at the top	Makes the models better over time using what everyone did, without knowing what any one person did.	A daily batch job. It aggregates events into counts, retrains the three levels, and ships a bundle. It is never called during a decision, so its latency and availability do not affect the commuter at all.

Note which way the arrows run. Nothing in the bottom half of the picture ever waits on anything in the top half.

3 What comes from the cloud

This is the part people ask about most, so it is worth being precise. The cloud contributes real value, and it does so without ever needing to know who anybody is.

Going up: what the phone sends

What is sent	What it looks like	What it is used for
Offer outcome counts	Bucketed tallies: *time-of-day band × area × offer category → shown 1,412, taken 61.* No user identifier, no journey, no exact location.	Rebuilding the popularity table that the **Light** models read from. This is what lets the simplest model still be sensible in a tunnel.
Content engagement counts	Same shape: *topic × article length band × time-of-day → opened, finished, skipped.*	Retraining the Standard and Strong feed models.
The switch log, aggregated	*Rule 2 fired 18,004 times yesterday; median time spent at Light was 4m 10s; 0.3% of switches were reversed within a minute.*	Checking the rules behave, and tuning the thresholds. This is about **our software**, not about people.
Health counters	Crashes, model load failures, decisions that timed out, blank-screen seconds.	Catching problems. Also feeds directly into the metrics in Section 6.

Coming down: what the phone receives

What arrives	How big	How often
The popularity table — what people broadly like, by hour and area. The Light models are a lookup into this.	about 60 KB	Nightly
Refreshed model files — new Light, Standard and Strong for both jobs.	about 2 MB for offers; the Strong feed model is larger and only downloads on wifi	Weekly, or when a retrain is worth shipping
Threshold updates — the numbers inside the six rules.	well under 1 KB	Rarely, and never mid-journey

All three arrive as one bundle, applied when the app is idle. A phone that has been offline for a fortnight runs on the bundle it already has, at slightly stale quality. Nothing breaks.

What the cloud is deliberately not

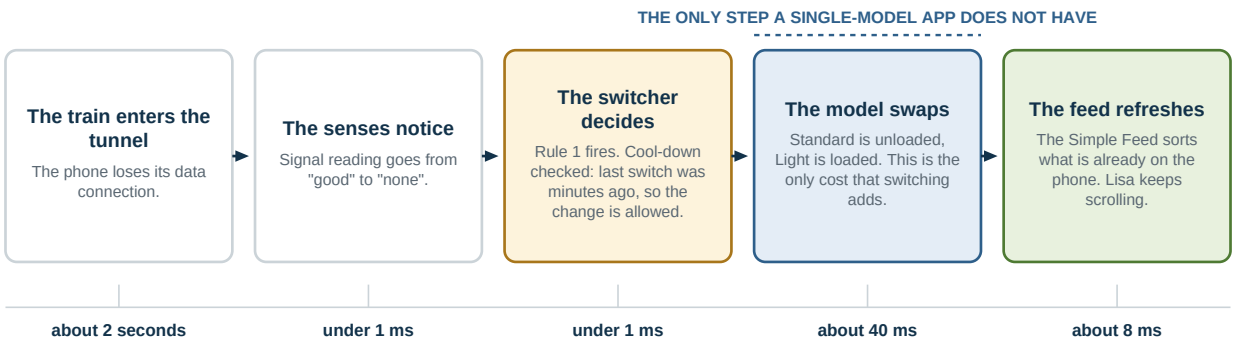
It is **not in the decision loop**. When the app picks an offer, it does not ask a server — that would defeat the whole point, because the moment we care most about is the one where there is no signal.

It does **not receive a per-person profile**. What goes up are counts that have already had the individual removed. Two people who did the same thing are indistinguishable in what we send, and a count of one is not sent at all.

It does **not send anything person-specific down**. The bundle that arrives on Lisa's phone is byte-for-byte the same one that arrives on John's. Everything personal about her recommendations is computed on her phone, from history that never left it.

4 One decision, end to end

Lisa's train enters the tunnel at 3:10. Here is the whole chain of events that follows, drawn to scale in time. It finishes before she could notice anything had happened.



The two-second wait at the start is deliberate: the app does not react to a single missed packet, it waits to be sure the signal is really gone. Everything after that is faster than a blink.

Total, from losing signal to a working feed: roughly two seconds, almost all of it the deliberate wait. The actual switching work is about fifty milliseconds.

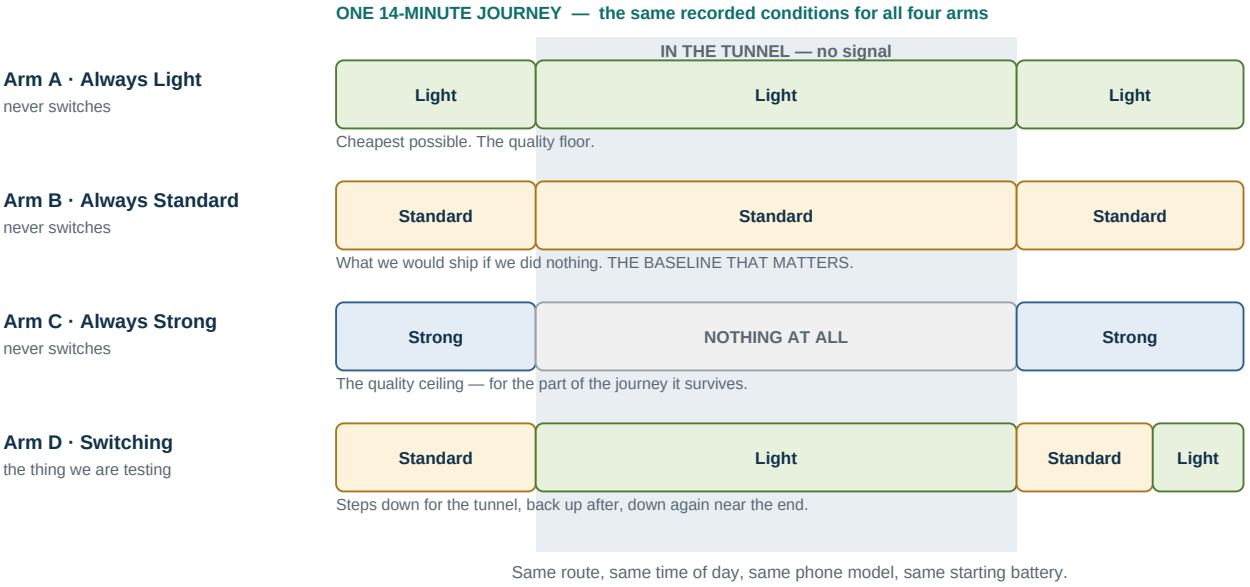
The step that costs us something

Everything in this chain except the fourth box also happens in an app with one fixed model. The swap — unloading one model and loading another — is the **extra** work that switching introduces, and it is the thing that could in principle make switching a bad deal. If we swap a handful of times a journey it is trivially worth it. If the rules were badly tuned and it swapped every few seconds, the swapping could easily cost more energy than the cheaper models save.

That is why **switch overhead as a share of the energy saved** is one of the headline numbers in Section 6, and not an afterthought.

5 How we will compare it

To say switching is better we have to say **better than what**. So we build four versions of the app and run them over the same journeys. Three of them never switch.



Arm B is the one that decides the project. Arm A tells us how much quality switching actually buys over doing the cheapest thing always; Arm C tells us how much quality we gave up against a ceiling that cannot actually be reached on a metro.

How the four arms get run

Stage	What happens	What it is good for
1. Replay	We take thousands of real recorded journeys — the signal, battery and timing traces, with the content of them stripped — and play the same trace through all four arms on the same bench phone.	Perfectly paired comparison. Every arm faces the identical fourteen minutes, so any difference in energy or coverage is caused by the arm and nothing else.
2. Watch-only on our own phones	Switching runs for real but changes nothing: the app records what it *would* have switched to, and keeps using Standard.	Every number about the switching behaviour itself — how often, which rules, whether it flaps — at zero risk to any commuter.
3. A live split	A small slice of real users, randomly assigned to an arm, for a few weeks.	The only way to get the numbers that involve human behaviour: redemptions, reading time, whether people come back.

Stages 1 and 2 give almost all of the energy and coverage evidence and cost nobody anything. Only stage 3 needs real users, and by then we already know the rules behave.

6 The metrics

Six families. The first two are the classic trade-off — did we spend less, did we lose quality. The third is the one only switching can win. The fourth asks whether the switching machinery paid for itself. The last two ask whether it helped a person, and whether it hurt anybody.

Family 1 — Energy and the phone

The "it cost less" claim. Measured on a bench phone with fixed screen brightness, so we are measuring the app and not the display.

Metric	Unit	Why it matters	What a win looks like
Battery used per journey	mAh per 14-min journey	The headline. What the commuter would actually feel.	Meaningfully below Arm B. We are aiming for at least 15–20% lower.
Energy per decision	millijoules per inference	Separates "cheaper model" from "we simply made fewer decisions".	Lower than Arm B, and the gap should track how much time was spent at Light.
Decision time	milliseconds, median **and** worst 5%	A good median with a terrible tail is a bad app. The tail is what people notice.	Both at or below Arm B. The worst 5% matters more than the median.
Peak memory	MB resident	Three models on the shelf could mean three models in memory, which would be a real cost.	No higher than Arm B — only the chosen model should be resident.
Thermal pressure	°C peak, and seconds spent throttled	A hot phone gets slowed down by the operating system, which makes everything worse.	Lower than Arm C. Ideally indistinguishable from Arm A.
Mobile data used	KB per journey	Strong models reach out to the network. Data costs the commuter money.	Well below Arm C. Close to Arm A.

Family 2 — Quality of what was shown

The "we did not lose anything" claim. This is the side we expect to be slightly worse, so the target is not "better" — it is "not meaningfully worse".

Metric	Unit	Why it matters	What a win looks like
Offer redemption rate	% of offers shown that get used	The one that makes money. If this drops, nothing else rescues the project.	Within a small, pre-declared margin of Arm B — we propose 2% relative.
Offer click rate	% of offers shown that get tapped	A faster-moving signal than redemption, useful while redemptions are still sparse.	Non-inferior to Arm B.
Agreement with Strong	% of decisions where the top pick was identical	The cheap, high-volume proxy. For every decision we can also ask what Strong would have said, offline, for free.	High — and crucially, high *in the moments we stepped down*. If Light disagrees with Strong 60% of the time in a tunnel, stepping down is expensive.
Regret	average drop in predicted value when it disagrees	Not all disagreements are equal. Picking the second-best offer is nearly free; picking the ninth is not.	Small and, more importantly, bounded — no long tail of very bad picks.
Ranking quality, offline	nDCG@5, Precision@3, AUC	Scored against logged outcomes. Lets us compare arms on millions of decisions without showing anything to anybody.	Arm D close to Arm C, clearly above Arm A.
Content engagement	open rate, reading seconds, % finished	The feed half of the story.	Non-inferior to Arm B, and clearly better than Arm C *in tunnels*, where Arm C shows nothing.

Why "non-inferior" and not "better"

The honest claim for model switching is not that it makes better recommendations. A smaller model is a smaller model. The claim is: **roughly the same quality, for meaningfully less energy, and available in far more moments**.

So the quality test has to be a *non-inferiority* test, with the acceptable margin written down **before** we look at any results. Deciding afterwards what counts as close enough is how projects talk themselves into shipping things that did not work.

Family 3 — Never blank

The family where switching does not compete with the baselines — it beats them outright. A fixed Strong model cannot answer in a tunnel, and the metro is mostly tunnel.

Metric	Unit	Why it matters	What a win looks like
Answered rate	% of decision requests that produced anything at all	The single clearest argument for the whole project.	Arm D at essentially 100%. Arm C will be far lower on a tunnel-heavy route.
Blank-screen seconds	seconds per journey with nothing in the feed	The commuter-facing version of the same thing. Seven blank minutes of a fourteen-minute trip is a broken app.	Near zero for Arm D. This is the number to put in front of anyone who is not convinced.
Time to first result	milliseconds, median and worst 5%	A result that arrives after they have put the phone away is not a result.	Better than Arm C, which has to wait on the network.
Timeout and error rate	% of decisions that failed or gave up	Distinguishes “could not answer” from “answered badly”.	Effectively zero for Arm D.

This family is worth stating separately because averaging it into a quality score hides it. An arm that gives brilliant answers 45% of the time and nothing the rest of the time can look respectable on a mean.

Family 4 — Did the switching itself pay for itself

The machinery has a cost. These are the metrics that would tell us switching was a bad idea, and they are the ones a sceptical reviewer will go to first.

Metric	Unit	Why it matters	What a win looks like
Switches per journey	count	Sanity. The design intends a handful.	Roughly 2–8. Dozens would mean the thresholds are wrong.
Flap rate	% of switches reversed within 60 seconds	Flapping is the classic failure of any rule-based switcher, and it burns energy for nothing.	Under 1%. The cool-down exists to make this true.
Cost of one switch	milliseconds and millijoules to unload and load	The price of the mechanism, measured directly rather than assumed.	Tens of milliseconds, and small enough that the next metric works out.
Switch overhead ratio	energy spent switching ÷ energy saved	**The make-or-break number.** If we spend more on swapping than we save by running smaller models, the entire idea is a loss regardless of every other metric.	Under 10%. Above 50% and we should stop and redesign.
Time at each level	% of journey at Light / Standard / Strong	Tells us whether the rules describe the world we thought they did.	No surprises. If Strong is never reached, Rule 6 is dead code.
Explainable switches	% of switches attributable to a named rule	If we cannot say why it switched, we cannot debug it or defend it.	Exactly 100%. Anything else is a bug, not a metric.
Gap to hindsight	quality vs a chooser that knew the future	Replay the journey and compute the best possible sequence of levels with full knowledge. That is the ceiling any rule set could reach.	A small gap means the rules are near-optimal; a large one means better rules exist and are worth finding.

Family 5 — Did it help a person

Everything above can look good while the app still gets deleted. These come from the live split only.

Metric	Unit	Why it matters	What a win looks like
Session length and frequency	minutes per journey; sessions per week	The broadest signal that the app is more useful than it was.	At or above Arm B.
Retention	% still active at day 7 and day 30	Slow but decisive. A feed that works in tunnels should show up here.	At or above Arm B, especially for tunnel-heavy routes.
App killed by the phone	background terminations per 100 journeys	The direct consequence of being a battery hog. It is also invisible unless you measure it.	Below Arm B and far below Arm C.
Crash and freeze rate	per 1,000 sessions	Loading and unloading models is exactly the kind of thing that introduces crashes.	No worse than Arm B. This is a gate, not a nice-to-have.
Battery complaints	mentions per 1,000 reviews; uninstalls after a long journey	Noisy, but it is the language real users actually use.	No increase.

Family 6 — Did it hurt anybody

Averages hide harm. Every metric above gets cut these four ways before we believe it.

Cut the results by…	The question it answers
Phone age and tier	Switching should *narrow* the gap between a new phone and a five-year-old one, because the old phone gets a model it can actually run. If the gap widens instead, we have built something that quietly works better for people with money.
How new the user is	Rule 4 steps new users down deliberately. We need to confirm that this protects them from confident bad guesses rather than just giving them a worse app for a month.
Route type	Tunnel-heavy versus overground. Almost all of the coverage benefit lives in the first group; if there is a cost, it will show up in the second.
The worst 5% of journeys	Not the average journey — the bad one. Low battery, old phone, long tunnel, new user, all at once. If switching helps anywhere, it should help most here.

If a headline number improves but one of these cuts gets worse, that is a finding, not a rounding error.

7 What a result looks like

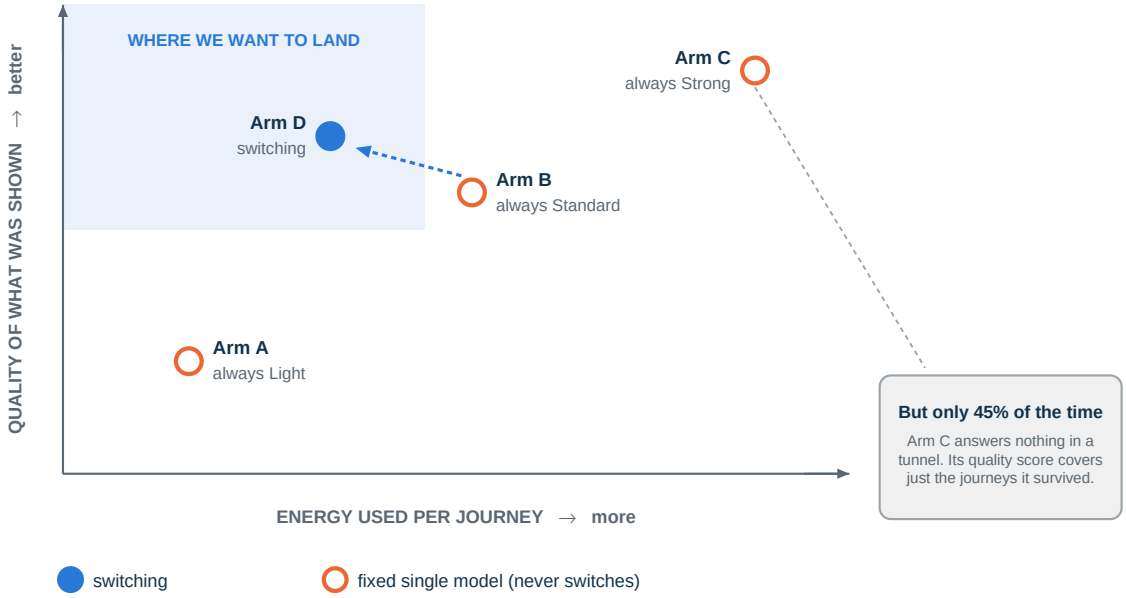
Sixty metrics do not make a decision. Four of them do. Everything else is there to explain the four, or to catch the case where one of them is lying.

	The condition	Measured as	Against
1	**It genuinely costs less.**	Battery used per journey, at least 15% lower.	Arm B — always Standard
2	**It is not meaningfully worse.**	Offer redemption rate within 2% relative, with the margin fixed in advance.	Arm B — always Standard
3	**It is available far more often.**	Blank-screen seconds per journey at or near zero.	Arm C — always Strong
4	**The mechanism paid for itself.**	Energy spent switching under 10% of the energy saved.	Itself

All four have to hold. Three out of four is not a pass — it is a specific, informative failure, and each one points at a different fix.

The picture we expect to end up with

Two of those conditions can be drawn together. Energy along the bottom, quality up the side. Every arm is one point, and the corner we want is **up and to the left**: as good, for less.



These positions are what we expect to find, not measurements — no data has been collected yet. *The diagram is here to make the shape of the argument clear: Arm D should sit up and to the left of Arm B, and the grey note is the reason Arm C sitting highest does not settle anything.*

Reading the result honestly

If Arm D lands where the dashed arrow points — clearly left of Arm B, at roughly the same height — switching worked, and the coverage numbers from Family 3 make the case stronger still.

If Arm D lands **below and left** of Arm B, we bought energy with quality. That is not automatically a failure, but it is a different, smaller claim and it has to be argued on its own terms.

If Arm D lands **right** of Arm B, the switching overhead ate the saving. Check Family 4 first — it is far more likely to be a flapping problem than a fundamental one.

8 Five ways to get this measurement wrong

These are the mistakes that produce a confident, well-presented, wrong answer. The first one is the one we are most likely to make.

	The trap	What it does to the numbers	How we avoid it
1	**Dropping the journeys where Strong could not answer.**	It is the natural thing to do — you cannot score a recommendation that never happened. But it silently removes every tunnel from Arm C's results, so Arm C gets judged only on its easiest moments and comes out looking excellent. This single decision could make switching look unnecessary.	A decision that produced nothing is scored as a **zero**, not as missing data. Every arm is scored over the same denominator: all decisions requested.
2	**Comparing different journeys.**	Arm C happens to get more overground routes, or sunnier weeks, or newer phones, and the comparison measures that instead of the arm.	Replay the identical recorded trace through all four arms on the same bench phone. For the live split, randomise per user and check the arms are balanced on route and device before reading anything.
3	**Letting the screen dominate the battery number.**	The display uses far more power than any model on the phone. A small difference in screen-on time between arms would swamp the entire effect we are trying to measure.	Fix screen brightness on the bench, and report the *app's share* of battery use rather than total drain. Report screen-on time separately as a check.
4	**Choosing the success margin after seeing the data.**	Whatever the quality drop turns out to be, it will be possible to describe it as acceptable. This is how a project convinces itself.	The 15% energy target and the 2% quality margin in Section 7 are written down now, before any measurement exists. If we later want to change them, we change them in writing and say why.
5	**Reporting only averages.**	A median decision time of 20 ms with a worst-5% of 900 ms is a bad app that looks like a good one. Averages also hide the fairness problems in Family 6 completely.	Every latency and energy metric is reported as median **and** worst 5%. Every headline is cut by phone tier, user age, and route before it is believed.

The cheapest thing we can do, and we should do it first

Watch-only mode. Switching runs, decides, and writes down what it *would* have done — and then does nothing. It gives us the whole of Family 4 (switches per journey, flap rate, time at each level, whether every switch is explainable) and a good estimate of Family 1, on real journeys, with real conditions, and with literally zero risk to any commuter.

If the rules are badly tuned, we find out here, for free, before anybody is affected.

9 Words we used

Mostly the measurement ones, since the architecture words are in the plain-English switching guide.

Word	What it means here
Arm	One version of the app in the comparison. We have four: always Light, always Standard, always Strong, and switching.
Baseline	The arm we are trying to beat. Ours is **always Standard**, because that is what we would ship if we did nothing at all.
Non-inferiority	Testing that something is *not meaningfully worse*, rather than that it is better. The right test when you are trading quality for cost, and it needs its margin agreed in advance.
Coverage / answered rate	The share of decisions that produced anything at all. A model that cannot run in a tunnel has poor coverage no matter how good its answers are.
Worst 5% (p95)	The value that only one journey in twenty is worse than. It is what people actually complain about; the median is what looks good in a slide.
Flapping	Switching back and forth because a reading is sitting right on a threshold. Burns energy and produces nothing.
Switch overhead	The energy and time spent unloading one model and loading another. The price of the mechanism, and the thing that could sink the whole idea.
Regret	How much worse the choice we made was than the choice the best available model would have made. Low regret means stepping down was nearly free.
Hindsight ceiling	The best any rule set could have done on a journey, computed afterwards with full knowledge of what happened. Tells us whether to invest in better rules.
Watch-only mode	The switcher decides and logs, but does not actually switch. Gives us most of the evidence at none of the risk.
Aggregated	Added up across many people until no individual remains. What goes to the cloud is counts, not records.
Replay	Playing a recorded journey — its signal, battery and timing trace — through each arm on a bench phone, so all four face identical conditions.

If you remember one thing

The architecture is simple: **three models of each kind sit on the phone**, a small rule engine watches four readings and picks one, and the cloud only sends down fresh files once a day from everybody’s behaviour added together.

The argument is equally simple, and it has three parts, not one: **less energy**, **about the same quality**, and **an answer in far more moments**. The third part is the one no single fixed model can match, and it is the part that is easiest to lose by measuring carelessly.